言語こそがプロンプトである：

Elixir が AutoCodeBench で Python をほぼ倍増させた理由と、3,920 タスクが明らかにする LLM に適した言語設計

Günther Brunner　グンター・ブルンナー

CyberAgent, Inc.（株式会社サイバーエージェント）

contact@guntherbrunner.com

https://guntherbrunner.com

2026 年 3 月

私の言語の限界は、私の世界の限界を意味する。

— ルートヴィヒ・ヴィトゲンシュタイン　『論理哲学論考』（1922 年）

Abstract

Erlang VM 上で動作するマイナー言語である Elixir は、AutoCodeBench で 87.4% の Pass@1 を記録した。一方、Python は 43.9% である。これは単なるリーダーボード上の珍事ではない。コード生成モデルが何を報酬として捉えているかを示す重要な手がかりである。本論文では、GPT-5.4（中推論）を用いて 20 言語による AutoCodeBench の結果を再現し、Elixir の優位性が難易度統制後も持続することを確認した上で、リーダーボードだけでは答えられない問いに踏み込む。すなわち、なぜこの結果が生じるのか、という問いである。

4 つの能動的アブレーション実験（計 2,970 条件評価）、144 行の完全一致タスク多言語パネル、および 384 行の分割要因計画による追加実験の結果、Elixir 内部で最も強い因果的シグナルはドキュメント構造であり、言語横断で最も強いポータブルな介入は明示的な契約記述であることが判明した。Elixir 内部では、構造化ドキュメントを除去するとパフォーマンスが約 40

パーセントポイント低下する。Elixir・Python・TypeScript にまたがる完全一致タスクにおいて、Holm 補正を生き残る唯一のポータブルな介入は明示的な契約記述（$\Delta = +0.359$、0–1 合格スケール）であり、Python（+0.469）と TypeScript（+0.453）で最大の効果を示した。

本研究ではこれらの結果を明示性仮説として定式化する。意図、契約、制御フロー、データ変換をローカルに可視化する言語とエコシステムは、コード生成モデルにかかる予測負荷を軽減する。Elixir は現行データにおける最強の事例であるが、本論文のより持続的で論争的な主張はさらに広い。すなわち、AI 支援プログラミングが当たり前になるにつれて、言語設計は性能やエルゴノミクスや人気だけでなく、機械にとってどれほど読みやすいかによっても評価されるようになるという主張である。

日本の読者の方へ　本論文の著者は株式会社サイバーエージェントに所属しており、本研究は日本のテック産業の文脈において特に重要な意味を持つ。Elixir は日本でも活発なコミュニティ（tokyo.ex、ElixirConf JP 等）を有し、日本企業での採用事例も増加している。Erlang および BEAM 仮想マシンの歴史は通信技術に根差しており、NTT をはじめとする日本の通信企業が培った技術基盤と深い関連がある。本論文の知見は、LLM を活用したソフトウェア開発において、言語選択とドキュメント設計が品質に直結することを実証的に示すものである。

キーワード — コード生成 · LLM 評価 · プログラミング言語 · Elixir · Python
ドキュメント設計 · ベンチマーク分析 · ソフトウェア工学

---

1 はじめに

まず、一つの印象的な数字から始めよう。87.4 である。

これは AutoCodeBench における Elixir の Pass@1 である。AutoCodeBench は 20 のプログラミング言語にまたがる 3,920 の自動生成コードタスクからなるベンチマークである。現代の機械学習において標準的な言語であり、公開コードコーパスにおいて最も多く表現される言語の一つである Python のスコアは 43.9% に留まる。JavaScript は 42.9% である。両者と比較して市場シェアが極めて小さい Elixir が、これらをほぼ倍増させているのである。

最初の直感的反応は「ノイズだろう」というものであろう。おそらく Elixir には簡単なタスクが

割り当てられたのではないか。おそらくベンチマークが異常に洗練されたドキュメントを持つリポジトリを過剰にサンプリングしたのではないか。おそらくこの効果はプロンプトスタイルや評価メカニクスの人工的産物ではないか。二番目の直感は、この結果を関数型プログラミングの勝利として扱うことである。しかし、いずれの直感も浅すぎる。効果は大きいが、その説明はより繊細である。

本論文は Tencent Hunyuan AI が公開した AutoCodeBench リーダーボード [1] を出発点とする。GPT-5.4（中推論）を用いて 20 言語の完全な評価を再現し、難易度統制後も Elixir が大きな正の外れ値であり続けることを検証した上で、リーダーボードの記述を超えて因果的調査へと進む。主要な知見は明確である。Elixir の優位性は、構文のミニマリズムや関数的純粋性だけでは最もよく説明できず、プログラムの意図とタスクの意味論をローカルなコンテキストから回復することを異常なほど容易にする設計上の決定の積み重ねと最も整合的なのである。

本論文の最も論争的な主張は、したがって、Elixir が普遍的に優れているということではない。それは、言語競争の次なる重要な軸がモデルにとっての可読性になりうるということである。契約、状態遷移、期待される振る舞いを明示する言語とエコシステムは、コード生成、コードレビュー、およびエージェント的ソフトウェア構築において構造的優位性を享受するであろう。

本研究の貢献

本論文は 6 つの具体的な貢献を行う。

1. **人工物の統制を伴う再現**。20 言語の AutoCodeBench 結果を再現し、難易度バケット統制および一言語除外期待値分析の後も Elixir が最大の正の外れ値であることを示す。
2. **能動的アブレーションによる因果的証拠**。4 つの完了したアブレーション実験（A, D, E, F）を通じて、Elixir 内部における支配的シグナルがドキュメント構造であることを明らかにする。サンプルの除去だけでは何も変わらないが、構造化ドキュメントを崩すと性能が約 40 パーセントポイント低下する。
3. **言語横断のポータビリティ分析**。144 行の完全一致タスクパネルおよび 384 行の分割要因計画による追加実験において、Holm 補正を生き残る唯一のポータブルな介入が明示的な契約記述であることを示す。
4. **新しい概念的定式化**。予測負荷フレームワークと可視性に基づく相互作用モデルを導入し、

Elixir のベンチマーク上の優位性と、すでに高い可視性を持つ言語における明示性の限界収穫逓減の両方を説明する。

5. **言語設計の再定義**。LLM にとっての親和性が実質的な設計軸になりつつあることを論じ、ドキュメントアーキテクチャがその軸の付随物ではなく第一級の構成要素であることを指摘する。
6. **プラットフォームレベルの含意**。明示性の物語を BEAM 仮想マシンのアクターモデルに接続し、Elixir をコード生成にとって読みやすくするのと同じ設計哲学が、エージェント的 AI 展開の要件とも整合する可能性を論じる。

構成。 第 2 節で関連研究を概観する。第 3 節でベンチマークの再現と人工物統制を詳述する。第 4 節で実験設計を概説し、第 5 節で言語設計空間をマッピングする。第 6 節で能動的アブレーションからの因果的証拠を提示し、第 7 節で完全一致タスク多言語研究を報告する。第 8–12 節で解釈的フレームワーク、ドキュメントアーキテクチャの比較、実践的含意、プラットフォームレベルの議論を展開し、第 13 節で結論を述べる。

2 関連研究

AutoCodeBench。 AutoCodeBench は、リポジトリの関数をマイニングし、自然言語タスクを生成し、公開・非公開テストを構築し、20 言語にわたるコード生成を評価する自動パイプラインを導入した [1]。その規模と多言語カバレッジは比較分析にとって特に有用であるが、同時に課題も生じる。タスクは言語間で共有されないため、リーダーボードの生のスコア差だけでは因果的な言語効果を分離できないのである。

関数型言語の評価。 FPEval は「おそらく関数型言語はコード LLM にとって単純に優れている」という仮説に対する独立した統制を提供する [2]。その研究では、より純粋な関数型言語は現行モデルにとって依然として困難であることが示されており、Elixir の結果が抽象的な関数的純粋性では説明できないことを示唆している。

低リソース言語の期待値。 低リソース言語に関する先行研究は、一般に本研究で観察されることの反対を予測する。すなわち、より小さな言語はプレトレーニングコーパスにおける表現が少な

いため性能が劣るはずである [3]。Elixir が Python を劇的に上回るという事実は、したがって特別な精査に値する。日本のソフトウェア開発現場においても、Python 偏重の傾向は強く、この知見は言語選択の再考を促すものである。

言語混同とポピュラリティバイアス。 最近の研究は、多言語コード生成におけるプログラミング言語の混同 [4] と、Python や馴染みのあるライブラリへの強いベースライン選好 [5] の両方を文書化している。この文脈において、Elixir の優越は二重に驚くべきものである。低リソースであり、かつ明らかな「デフォルト」言語でもないにもかかわらず勝利しているのである。

コミュニティの観察。 Elixir コミュニティはベンチマーク結果にいち早く注目し、実務者のコメンタリーではドキュメント品質、ローカル推論、エコシステムの一貫性が候補的説明として強調されてきた [6, 7]。日本の Elixir コミュニティ（tokyo.ex 等）でもこの結果は大きな関心を集めている。本論文はそうした直感を統制された実証分析へと昇華させる。

BEAM ランタイムとエージェント的 AI。 ベンチマークの物語とは独立に、最近の研究は BEAM アクターモデルと新興の AI エージェントアーキテクチャの間の構造的類似性に注目している。Guimarães は、Erlang が 1986 年に導入したアクターモデルが LLM オーケストレーションのために再発見されつつあるエージェントモデルと密接に類似することを論じている [18]。NVLang は BEAM 言語のコード生成タスクにおける性能についての追加的証拠を提供する [19]。これらの接続について、第11節で慎重に再検討する。なお、Erlang と BEAM の技術は、日本では NTT の通信システム研究を通じて早い段階から注目されてきた歴史がある。

3 ベンチマークと主要結果

3.1 リーダーボード

表1は再現された 20 言語リーダーボードの上位 5 言語と下位 5 言語を示す。図1は完全なランキングを提示する。

特に断りのない限り、本論文におけるリーダーボードの数値は、修正されたホストネイティブスコアリングパスを用いた 2026 年 3 月 11 日のローカルフォーク再現を指す。このスナップショ

ットは合計 3,920 行を保持するが、均一な 196 タスクではなく、言語ごとにわずかに不均等な合計を持つ。Elixir の再現スライスは 198 タスクを含み、後述の能動的アブレーション実験も同一の 198 タスク Elixir スライスを再利用する。

Table 1. Pass@1 による上位 5 言語と下位 5 言語（2026 年 3 月 11 日のローカル AutoCodeBench 再現）。

順位	言語	Pass@1	難問率
1	Elixir	87.4	70.2
2	Kotlin	76.5	43.5
3	C#	72.4	46.2
4	Ruby	63.0	54.0
5	Julia	57.0	62.5
16	Python	43.9	68.4
17	Swift	43.5	60.5
18	Go	42.9	61.3
19	JavaScript	42.9	64.1
20	PHP	35.7	55.8

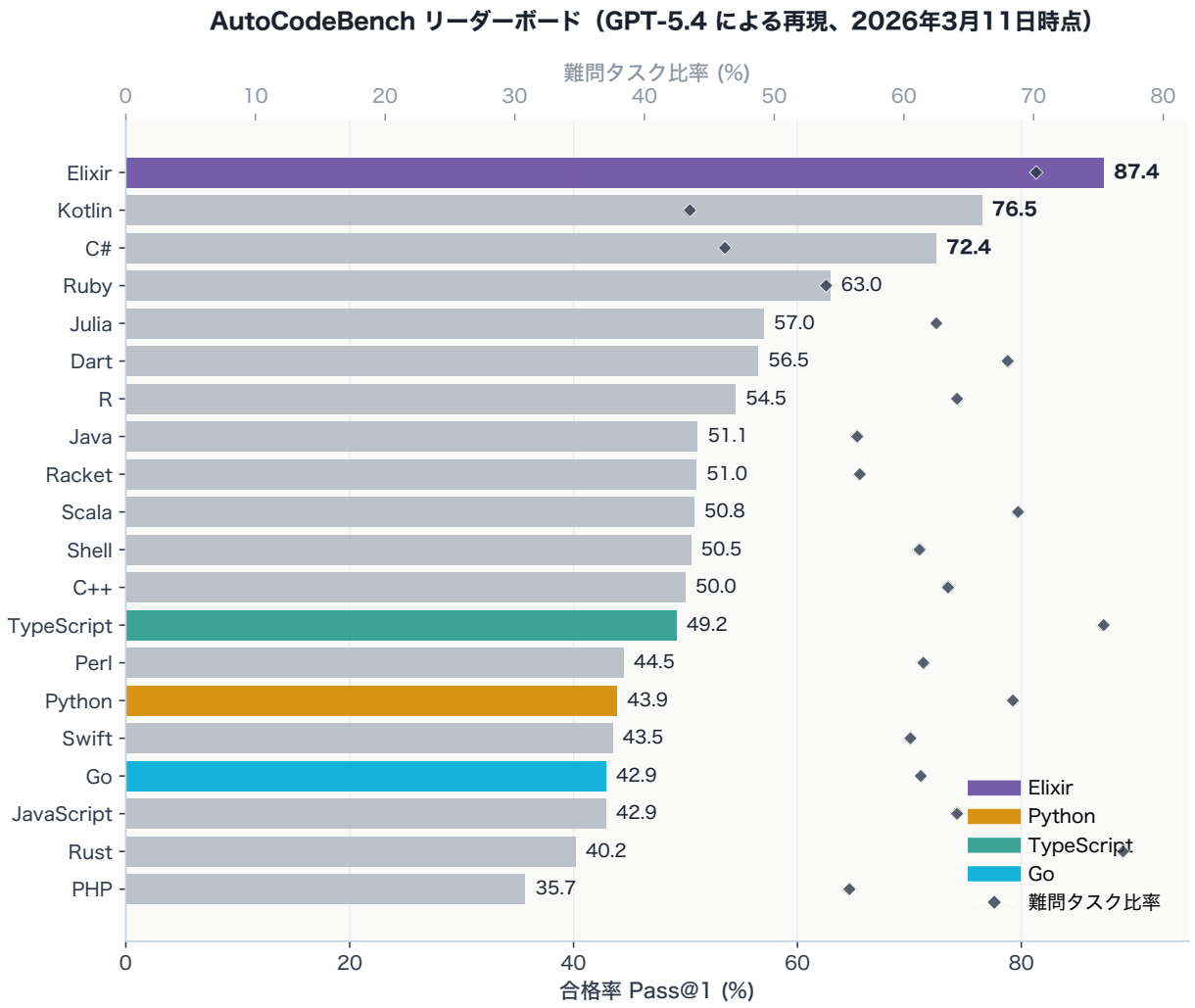

Figure 1. 全 20 言語リーダーボード。棒グラフは Pass@1 を示し、重畳されたマーカーは各言語の難問タスクの割合を示す。Elixir は最高性能であると同時に、最も重い難問タスク構成を持つ言語の一つである。

第一に重要な点は、これが僅差の勝利ではないということである。Elixir は 2 位の Kotlin を 10.9 pp、Python を 43.5 pp 引き離している。第二に重要な点は、この優位が特に容易な問題を受け取ったことによるものではないということである。Elixir の難問タスク比率はベンチマーク中央値を上回っている。

Go との比較。 特に示唆に富む定性的対比は Go とのものである。両言語とも並行性を重視したサーバーサイドのワークロードをターゲットとし、両者とも明示的なエラーハンドリングを強調

する（Go は複数戻り値、Elixir はタグ付きタプル）。両者ともに比較的小さなコア構文と直接的な操作モデルを好む。しかし Go のスコアは 42.9% であり、JavaScript と同一で Elixir に 44.5 pp 及ばない。本論文では Go のドキュメントアーキテクチャを直接測定していないため、この比較は結果ではなく仮説として扱う。Go–Elixir のギャップは、個々の関数レベルでの明示性は必要だが十分ではなく、契約、ドキュメント、サンプルのアーキテクチャ的統合が個別の機能よりも重要であるという考えと整合的である。

3.2 難易度耐性

表2と図2は、5 つの参照言語の難易度バケット別内訳を示す。

Table 2. 難易度バケット別の主要言語の Pass@1。

言語	易	中	難
Elixir	96.6	86.7	86.3
Kotlin	100.0	88.1	63.6
C#	97.8	81.1	63.1
Python	82.0	48.6	31.6
JavaScript	78.6	43.7	31.0

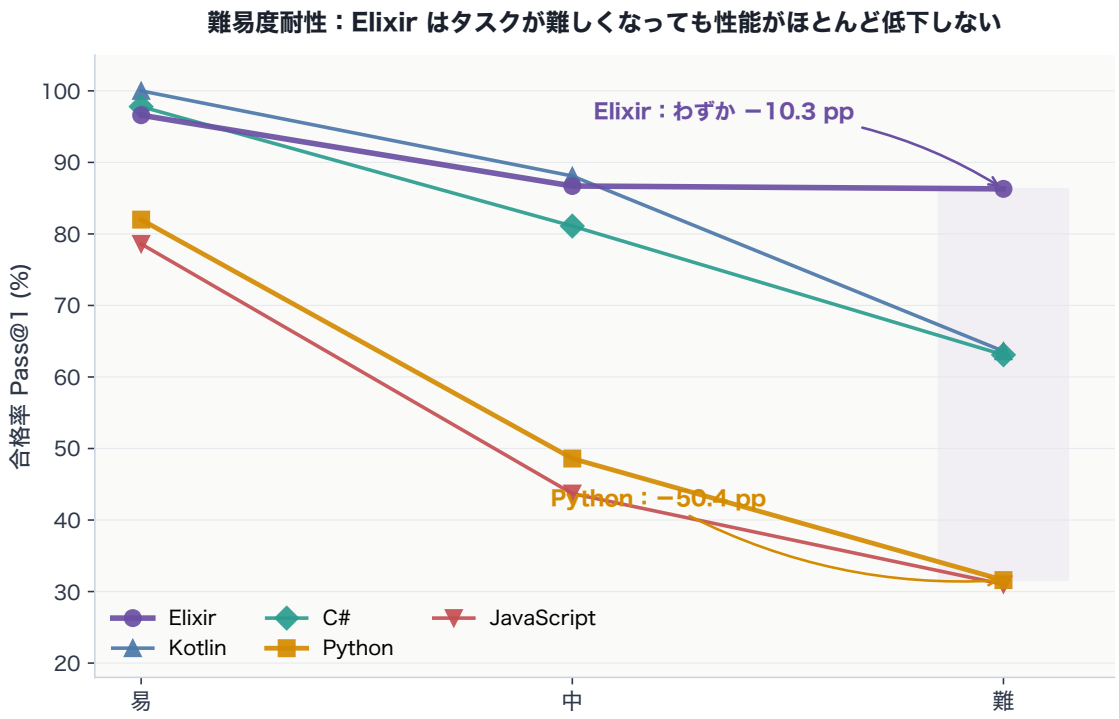

Figure 2. Elixir の優位性は、脆弱な生成が失敗するはずの難問タスクにおいて最も顕著である。Python は易から難にかけて 50.4 pp 低下するのに対し、Elixir はわずか 10.3 pp の低下に留まる。

これは質的に重要なパターンである。Elixir は単に高い出発点を持つのではなく、タスク難易度の上昇に伴う劣化が異常に小さいのである。これは、偶然のベンチマーク上の癖から恩恵を受ける言語・タスクの組み合わせではなく、曖昧さを低減する言語・タスクの組み合わせから期待されるパターンと一致する。日本語で平たく言えば、Elixir は「難しい問題ほど差が開く」のである。

3.3 人工物の統制

リーダーボードの差がタスク構成によって説明可能かどうかを検証するため、各言語について難易度バケット内の他 19 言語の合格率を用いて期待成功率を計算した：

$$\hat{p}_{\text{expected}}(L) = \sum_{d \in \{E,M,H\}} \frac{n_d(L)}{N(L)} \cdot \bar{p}_d(\neg L). \tag{1}$$

Elixir の場合、観測された 87.4% は難易度調整済みの期待値 44.7% を 42.7 pp 上回る。これはデータセット中で最大の正の残差である。

4 実験設計と方法論

4.1 仮説検証スイート

調査を 9 つの仮説検証スイートに組織した。その概要を表3に示す。

Table 3. Elixir の優位性を調査するために使用した仮説検証スイート。

スイート	次元	手法	状態
A	ドキュメントとタスクフレーミング	Elixir タスクにおけるプロンプト側ドキュメントの除去・劣化	完了
B	パターンマッチングの範囲	20 言語横断のプロキシメトリック	プロキシのみ
C	パイプライン合成	20 言語横断のプロキシメトリック	プロキシのみ
D	制御フロースタイル	パターンディスパッチ構造の代替形式への書き換え	完了
E	エラーハンドリング慣習	タグ付きタプルの代替戻り値慣習への置換	完了
F	状態とデータフロー	パイプライン、再束縛、状態スレッディングスタイルの変更	完了
G	型システムシグナル	仕様とアノテーションからのプロキシメトリック	プロキシのみ
H	難易度構成の人工物	一言語除外期待値分析	完了
I	リポジトリスケールのエコシステム効果	フルプロジェクト評価	今後の課題

スイート A, D, E, F は計 792 のスイート・タスクスロットと計 2,970 の条件レベル評価をカバーする。スイート H は核心的な人工物統制である。スイート B, C, G は因果的主張ではなく記述的文脈を提供する。

能動的アブレーション実験は、リーダーボード再現に使用したのと同一のローカルフォークス

ナップショットから抽出した固定 198 タスク Elixir スライスに対して実施する。このマッチドスライス設計は意図的なものである。スイート内のすべてのデルタは同一タスク上の条件を比較するが、結果として得られる `full_docs` ベースライン（84.3%）は 87.4% のフルベンチマークスコアの第二の推定値として読むべきではない。

4.2 プロキシメトリック

20 言語を比較するために 3 つの要約メトリックを使用する。

制御フロー明示性

$$S_{\mathrm{cf}}(L) = \alpha \bar{P}(L) + \beta \bar{D}(L) - \gamma \bar{B}(L), \tag{2}$$

ここで $\bar{P}(L)$ は平均パターンマッチ密度、$\bar{D}(L)$ はパイプラインまたはディスパッチ密度、$\bar{B}(L)$ は分岐密度である。

可変性負荷

$$M(L) = \bar{A}(L) + \bar{U}(L) + \bar{W}(L), \tag{3}$$

ここで代入、更新、書き込みはすべて、モデルが追跡しなければならない隠れた状態に寄与する。

初回失敗分類法各失敗生成を初回の非合格状態で分類する：構文またはコンパイルエラー、実行時例外、誤出力、タイムアウト、またはその他の実行時失敗。Wilson 区間と Fisher 正確検定を割合比較に使用する。

スイート A, D, E, F の条件効果はマッチドタスク結果から推定し、すべての非ベースライン介入コントラストに対して Holm 補正付きの両側正確 McNemar 検定で検定する。384 行の要因計画では、各主効果をマッチド平均デルタとして要約し、タイを除外した勝ち/負けカウントに対する両側正確符号検定で検定し、4 つの主効果に Holm 補正を適用する。

4.3 主張の階層

修辞的な過剰主張を避けるため、本論文は主張を証拠の階層によって区分する。

Table 4. 主張レベルの要約。

#	主張	証拠レベル	主要な証拠
1	Elixir の 87.4% は人工物統制を生き残る	強い	難易度統制と +42.7 pp 残差
2	ドキュメント構造が Elixir 内部の支配的要因	強い(因果的)	スイート A：約-38〜-41 pp
3	パターンマッチングディスパッチスタイルは独立に有意ではない	強い(帰無)	スイート D：全効果 ±3 pp 以内
4	エラーハンドリングスタイルは独立に有意ではない	強い(帰無)	スイート E：全効果 ±3 pp 以内
5	状態フロースタイルは示唆的だが未確立	方向性のみ	スイート F：+5.1〜+5.6 pp
6	明示的契約が主要なポータブルな言語横断介入	中程度(因果的)	要因計画：$\Delta = +0.359$、Holm 調整 $p = 0.030$
7	明示性仮説は有用な統合的解釈である	フレームワーク	完了した全証拠を統合

5 設計空間分析

能動的アブレーションに移る前に、Elixir が 20 言語の設計空間においてすでに異常に見えるかどうかを確認しておくことが有用である。図3はその問いにイエスと答える。

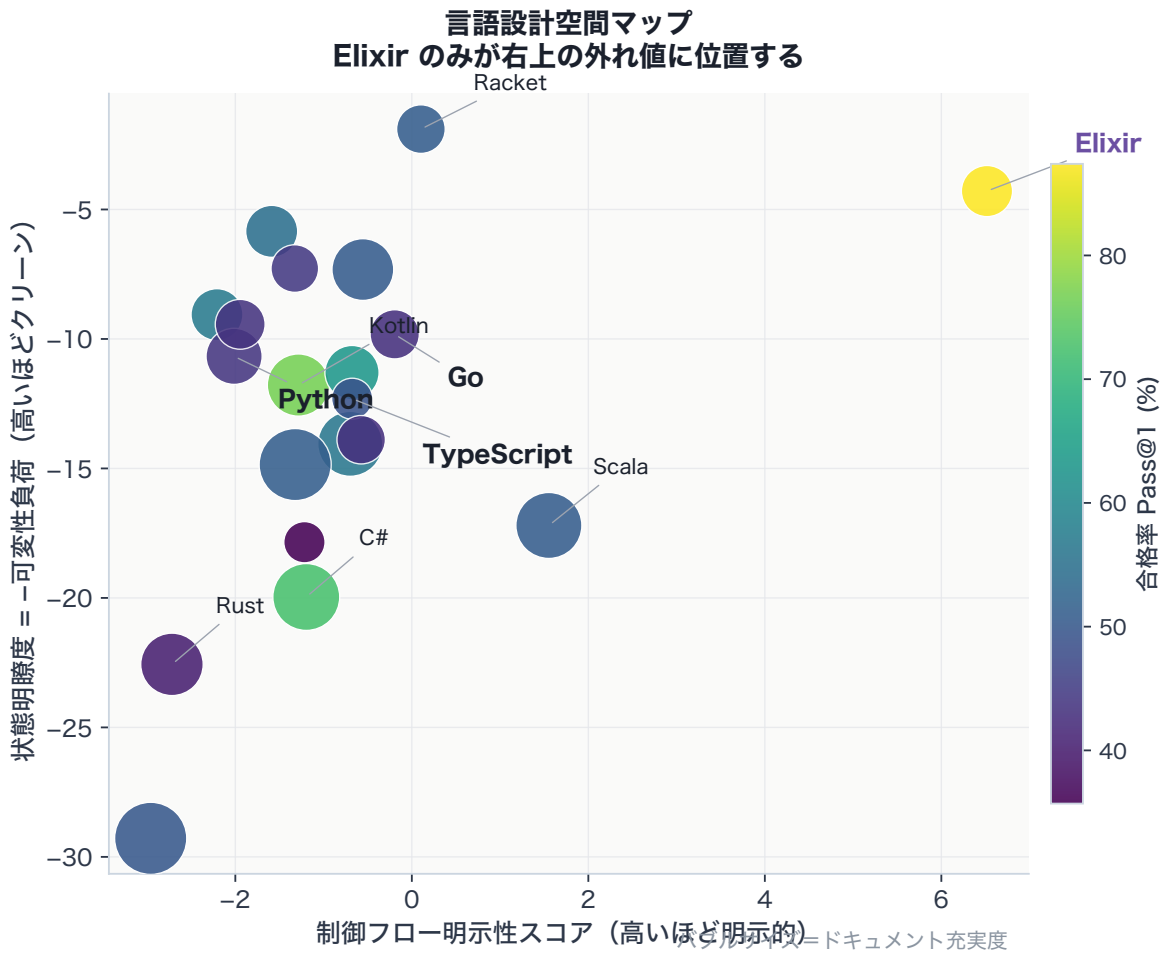

Figure 3. 20 言語プロキシテーブルから構築した設計空間マップ。横軸は制御フロー明示性を測定し、縦軸は状態明瞭度（可変性負荷の符号を反転させ、高いほど良いとしたもの）を示す。バブルサイズはドキュメントプロキシを反映する。Elixir は唯一の「右上方の遠い外れ値」である。

ここで 3 つの記述的事実が重要である。

第一に、Elixir は強い正の制御フロー明示性スコア（+6.517）を持つ唯一の言語である。第二に、その可変性負荷はデータセットの下位近くにある。第三に、そのドキュメントプロキシは一意に最大というわけではない—Java や C++ は生のボリュームでは上回る—ことから、量よりも構造が重要であることがすでに示唆されている。

プロキシ分析は因果関係を確立しないが、より正確なターゲットを示唆する。Elixir は明示的なディスパッチ、低い隠れ状態、およびコード自体と異常なほど統合されたドキュメント文化を兼ね備えているのである。

6 能動的アブレーションからの因果的証拠

6.1 スイート A：ドキュメント構造が支配的要因

本論文で最も強い結果はスイート A であり、表5と図4に示す。スイート A は前述の固定 198 タスク Elixir アブレーションスライスを使用するため、その `full_docs` ベースラインは 87.4% のフルベンチマーク値ではなく、マッチドスライスベースラインである。

Table 5. スイート A 能動的アブレーション（各条件 198 タスク）。

条件	合格数	Pass@1	全ドキュメントとの差
full_docs	167	84.3%	—
reference_no_examples	167	84.3%	0.0
minimal_docs	91	46.0%	-38.4
signature_only	85	42.9%	-41.4

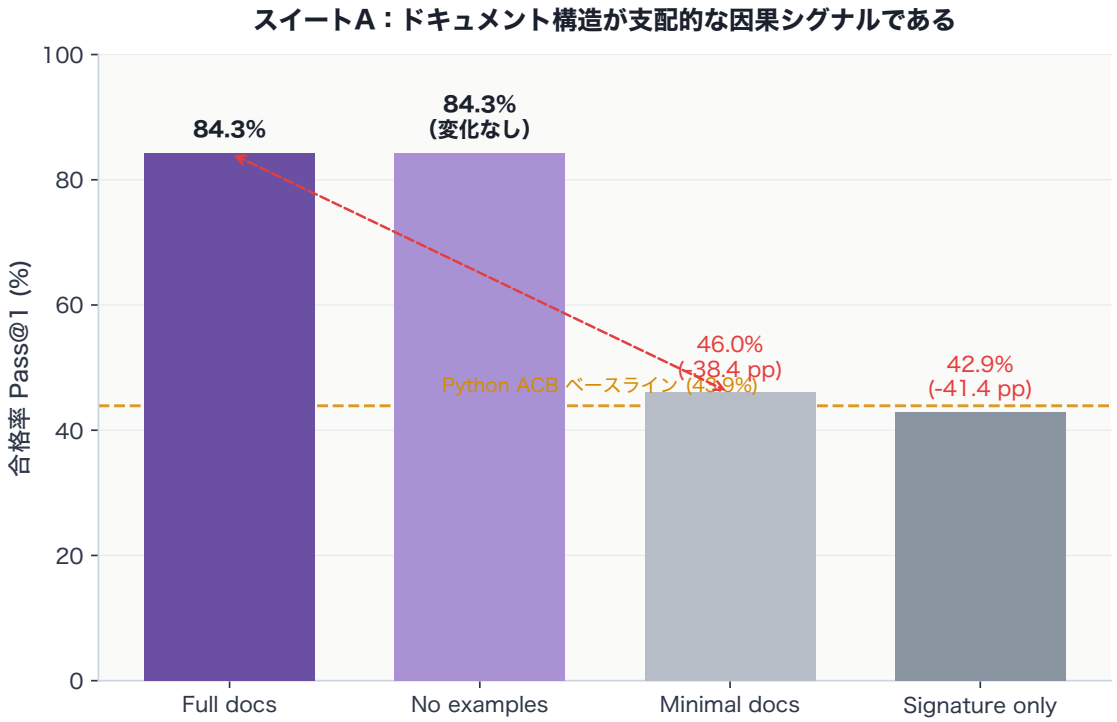

Figure 4. Elixir 内部において、サンプル単独では影響がないが、ドキュメント構造を崩すと Pass@1 がほぼ半減する。シグネチャのみのフレーミングでは、Elixir は Python の ACB 全体ベースラインの近傍まで低下する。

3 つの点が強調に値する。

1. **サンプル単独ではベンチマークを動かさない**。完全な Elixir フレーミングからサンプルを除去しても何も変わらない。
2. **構造こそがすべてである**。構造化ドキュメントの除去は性能を約 40 pp 低下させる。
3. **量は鍵となる変数ではない**。`minimal_docs` と `signature_only` の差は小さい。少しの非構造化テキストでは不十分である。重要なのはローカルに完全な仕様であること。

両方の大きな低下は、全能動的アブレーションコントラストにわたるマッチド McNemar 分析において Holm 有意である。

このパターンが本論文の中心的な因果的結果である。これは Elixir の問いを「モデルはどんな構文を好むか？」から「モデルは他を探さずにどんな情報を回復できるか？」へと再定義するものである。これは日本のソフトウェア開発においても重要な示唆を含む。すなわち、関数のドキュ

メントを「あとで書く」のではなく、コードと一体的に構造化して管理することが、AI 時代のコード品質を左右するのである。

6.2 その他の Elixir 内スイート

スイート D, E, F は物語を覆すのではなく、先鋭化する。制御フローをパターンディスパッチ構造から書き換えても大きなまたは有意な効果は生じない。エラーハンドリング慣習の書き換えも同様である。代替的な状態フロースタイルは小さな正のデルタを示すが、多重比較補正を生き残らない。完全な要約は付録Bに示す。

この帰無的証拠は価値がある。最良の説明が単一の崇拝された言語機能ではないことを示しているからである。Elixir の優位性は「パイプ演算子が勝った」のでも「タグ付きタプルが勝った」のでもない。シグナルは意味論的スタックのより高い位置に集中している：ドキュメント、契約、タスクフレーミングである。

6.3 失敗分類法とコモンタスクの健全性チェック

198 タスクの全ドキュメント評価においては 25 件の失敗のみが存在する。そのうち 7 件が誤出力、2 件が言語レベル例外、16 件がその他のランタイムまたは環境の失敗である。例外カウントの非常に少ないことは、多くの制御パスを例外駆動ではなく明示的にする言語と少なくとも方向性において整合的である。

再帰タスクの固定効果健全性チェックも同方向を指す。Elixir が登場する再帰タスクタイトルの小さなサブセットにおいて、クラスター平均に対する平均残差は正（+0.2143）であるが、クラスター数が少ないため区間は広い。これはトピック構成が無関係であることを証明しないが、最も強い純粋に観察的な反論を弱めるものである。

7 完全一致タスク多言語研究

7.1 144 行完全一致タスクパネル

メインリーダーボードは異なるタスクで言語を比較する。この制約を部分的に中和するため、144 行の完全一致タスクパネルを構築した：16 の手作成の実行可能タスクを、Elixir・Python・

TypeScript のそれぞれ 3 つのフレーミング条件で実装したものである。

集約した条件別結果は以下の通りである：

条件	Pass@1（48 行）
baseline_compact	35/48 = 72.9%
rich_contract	35/48 = 72.9%
rich_contract_examples	40/48 = 83.3%

2 つの点が重要である。

第一に、すべての言語に同一のタスク仕様が与えられた場合、Elixir はもはや支配的ではない。これは重要である。AutoCodeBench の結果が言語特性とベンチマークのネイティブなフレーミングスタイルとの相互作用の一部であることを示唆するからである。

第二に、この完全一致タスク設定ではサンプルが方向性として有用になる：マッチド改善 7 件対悪化 2 件。これはスイート A と矛盾しない。スイート A ではサンプルがすでにリッチで構造化されたフレーミングから除去された。完全一致タスクパネルではサンプルがコンパクトな仕様に追加されたのである。

7.2 384 行分割要因計画による追加実験

最も情報量の多い同一タスク結果は 384 行の正則 $2^{(4-1)}$ 分割要因計画から得られる。これは 4 つのプロンプト側要因を変動させる：リッチな散文、サンプル、明示的契約、状態フローガイダンスである。主効果はマッチドデルタとして推定し、タイを除外した勝ち/負けカウントに対する両側正確符号検定で検定し、4 要因にわたる Holm 補正を適用する。

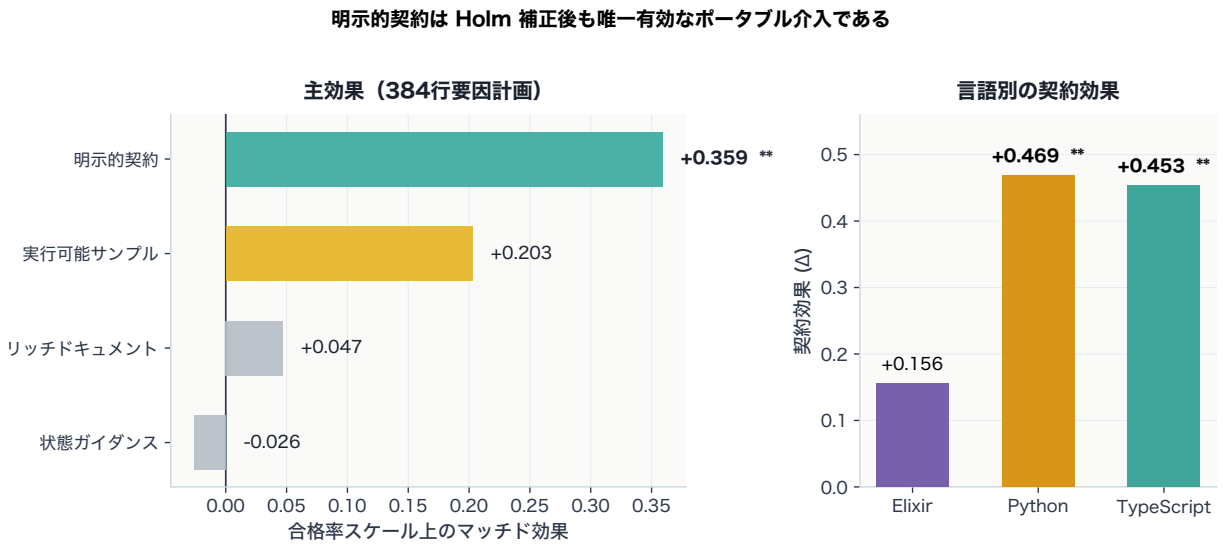

Figure 5. 完全一致タスク要因計画において、明示的な契約記述は Holm 補正を生き残る唯一のポータブルな介入である。その効果は Python と TypeScript で最大であり、これらは類似の契約情報が周辺のエコシステムにあまりネイティブに存在しない言語である。

Table 6. 384 行分割要因計画における主効果。

要因	マッチド Δ	正確符号 p	Holm 調整 p
docs_rich	+0.047	0.549	1.000
examples	+0.203	0.039	0.116
contracts_explicit	**+0.359**	**0.007**	**0.030**
state_guidance	-0.026	0.727	1.000

言語別の契約効果は以下の通りである：

言語	契約 Δ	Holm 調整 p
Elixir	+0.156	0.774
Python	**+0.469**	**0.001**
TypeScript	**+0.453**	**0.004**

これはまさに可視性に基づく説明が予測するパターンである。明示的な契約記述から最も恩恵を受ける言語は、そのエコシステムがその情報を Elixir ほど積極的かつローカルに表面化させない言語なのである。

換言すれば、Elixir は「すでに契約が見えている」ため追加契約の効果が小さく、Python と TypeScript は「契約が見えにくい」ため追加契約の効果が大きい。これは日本のソフトウェア開発チームにとって直接的な教訓を含む。たとえ Elixir を使わなくとも、関数の入出力契約を明示的に記述することで LLM によるコード生成品質を大幅に向上させることができるのである。

8 考察：予測負荷、可視性、ベンチマーク整合性

実証的な全体像は、定式化に足るほど明瞭になった。データの最良の解釈は単一の機能の物語ではなく、可視性の物語である。

8.1 ローカルな意味論的可視性

次のように定義する：

$$V(L,T) = \lambda_t E_{\text{task}} + \lambda_c E_{\text{contract}} + \lambda_f E_{\text{flow}} + \lambda_d E_{\text{dispatch}}, \tag{4}$$

ここで各 E 項は言語 L とタスク T について異なる意味論的次元のローカルな可視性を捕捉する：

- E_{task}：タスク記述が目的とエッジケースをどれほど明確に述べているか
- E_{contract}：入出力とエラー契約がどれほど明確に記述されているか
- E_{flow}：状態遷移とデータ変換がどれほど可視的であるか
- E_{dispatch}：制御フロー選択がどれほど明示的であるか

次に、成功を概念的に以下のようにモデル化する：

$$\Pr(\text{pass} \mid L, T) = \sigma(\alpha + \beta V(L, T) - \kappa D(T)), \tag{5}$$

ここで $D(T)$ は内在的タスク難易度であり、$\sigma(\cdot)$ はロジスティック関数である。

式(4)–(5)は解釈的なものであり、フィットされた推定値ではない。その価値は説明力にある。これらの式は、Elixir 内アブレーション、完全一致タスク多言語追加実験、および複数の低レベル機能書き換えの弱い役割を統合的に説明できるメカニズムの種類を教えてくれる。

8.2 ベンチマーク整合性

AutoCodeBench の完全な結果は主効果以上のものを示唆する。それは言語設計とベンチマークフレーミングの豊富さの間の相互作用を示唆する：

$$\Pr(\text{pass} \mid L, T) = \sigma(\alpha + \beta V_L + \gamma R_T + \delta V_L R_T - \kappa D_T), \tag{6}$$

ここで V_L は言語レベルの可視性事前分布であり、R_T はタスクフレーミングの豊富さである。

この相互作用モデルは、本論文における最も重要な 2 つの実証的事実を直ちに説明する。

- ネイティブな AutoCodeBench タスクでは、Elixir は V_L とベンチマークのフレーミングスタイルの両方が有利であるため大差で勝利する。
- 完全一致タスクパネルでは、フレーミングが言語間で統制・均等化されるため、Elixir は同一の固有な相互作用ボーナスをもはや享受しない。

したがって、本論文の真の教訓は「Elixir を使え」よりも広い。ベンチマークは整合された明示性を報酬として与えているように見える。

8.3 飽和の結果

Elixir と Python/TypeScript の間の契約効果の非対称性も、可視性モデルから自然に導出される。

命題 1 (明示性向上の飽和). 式(5)の下で、任意の可視性次元 E_k を改善することによる限界利得は

$$\frac{\partial}{\partial E_k} \Pr(\text{pass} \mid L, T) = \beta \lambda_k P(1 - P), \tag{7}$$

ここで $P = \Pr(\text{pass} \mid L, T)$ である。したがって、同一の可視性改善はベースライン性能がすでに天井に近い場合により小さな絶対利得をもたらす。

証明の概略。式(5)のロジスティック形式を E_k で微分する。乗数 $P(1-P)$ は $P = 0.5$ 付近で最大であり、$P \to 0$ または $P \to 1$ で縮小する。

これは実証的に重要である。Elixir はすでに強いローカル契約可視性を出発点として持つため、同じ明示的契約介入はベースラインの契約可視性が弱い Python や TypeScript よりも小さな限界利益しかもたらさない。数学的には限界収穫逓減の法則であり、直感的には「すでに十分見えているものをさらに見えるようにしても効果は小さい」ということである。

8.4 メカニズムの要約

Table 7. メカニズムレベルの証拠要約。

メカニズム	Elixir 内の証拠	言語横断の証拠	判定
ドキュメント構造	+38〜+41 pp、マッチド McNemar 検定で Holm 有意	ポータブルには未検証	**支配的要因**
契約の明示性	ベースラインがすでに高い	+0.359、正確符号検定で Holm 有意	**ポータブルな介入**
制御フロースタイル	≤ 3 pp、帰無	分離されていない	主要因ではない
エラーハンドリング	≤ 3 pp、帰無	分離されていない	主要因ではない
状態フロースタイル	+5.1〜+5.6 pp、非有意	分離されていない	示唆的のみ

8.5 情報密度との関連

可視性の説明は、人間の言語処理における均一情報密度の概念とも自然に接続する [8]。分散的でローカルに利用可能な情報は、まばらで高サプライズのバーストで到着する情報よりも処理しやすい。コードにおいて、パターンディスパッチヘッド、明示的パイプライン、近接する契約、実行可

能なサンプルはすべて同じ一般的効果を持つ：意味論的サプライズを平坦化するのである。本研究でトークンレベルのエントロピーを測定したとは主張しないが、理論的接続は直接的なフォローアップ研究を動機づけるのに十分強い。

9 ドキュメントアーキテクチャ：なぜ Elixir は構造的に異なるのか

実証結果は繰り返し上方を、すなわち構文からアーキテクチャへと指し示す。Elixir のドキュメントスタックの何が正確に特別なのか。

Table 8. ドキュメントアーキテクチャの比較。

特徴	Elixir	Python	TypeScript
ソース内モジュールドキュメント	`@moduledoc` 属性	モジュール docstring	組込みの第一級相当物なし
ソース内関数ドキュメント	`@doc` 属性	関数 docstring	JSDoc または TSDoc コメント
実行可能サンプル	組込みの doctest ワークフロー	標準ライブラリ doctest は存在するが使用が不均一	ネイティブ相当物なし
型と文書の近接	`@spec` がドキュメントに隣接	型ヒントはしばしば散文から分離	型はしばしばコメントから分離
ドキュメント生成	ExDoc	Sphinx, pdoc 等	TypeDoc 等
一元的ホスティング規範	HexDocs	断片的、任意参加	断片的、任意参加
ランタイム検索	`Code.fetch_docs/1`、IEx help	`help()` via pydoc	エディタおよびビルドツール中心

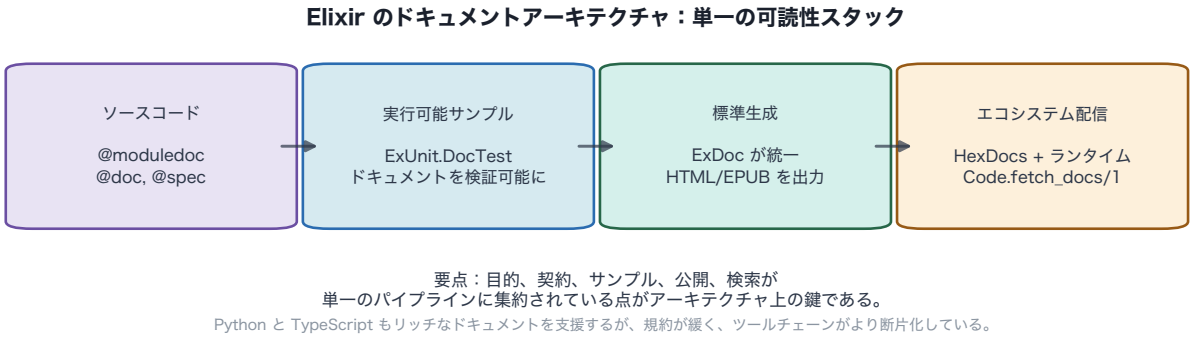

Figure 6. Elixir の優位性はスタイルではなくアーキテクチャにある。言語、テスティングフレームワーク、ドキュメント生成器、パッケージエコシステム、ランタイムヘルプ機能が単一のドキュメントパイプラインを形成する。

鍵となる違いは、Elixir がより多くの散文を持つことではない。生のドキュメント量はこの結果を説明しない。違いは、目的、サンプル、契約、公開、検索がすべて安定したパイプラインの中に共存していることである。ソースレベルのドキュメント属性、実行可能な doctest、ExDoc による生成、HexDocs での公開、IEx または `Code.fetch_docs/1` によるランタイム検索が一つのシステムの一部である [9–13]。Python と TypeScript も確かにリッチなドキュメントをサポートするが、それはよりゆるい慣習とより断片化されたツールを通じてである [14–17]。

その構造的統合が重要なのは、コード生成モデルがコンテキスト制約の下で動作するからである。関数の説明が物理的にも意味論的にも近接している場合–別のファイル、別のツール、別の社会的慣習に隠れているのではなく–モデルは恩恵を受ける [20]。

日本のソフトウェア開発の文脈で考えると、これはドキュメントを「コードの付随物」ではなく「コードアーキテクチャの一部」として設計すべきことを意味する。日本企業では仕様書や設計書を重視する文化があるが、それがコードと物理的に分離している限り、LLM にとっての可視性は限定的なのである。

9.1 これが単なる Elixir 礼賛ではない理由

この主張の健全な版はポータブルである。もし明示的契約が Python と TypeScript で最大の言語横断的効果を生むなら、本論文はチームにスタックを Elixir で書き直せと言っているのではない。すでにいる場所で予測負荷を減らせと言っているのである：

- 入出力契約を明示的に記述する
- サンプルをそれが説明するコードに隣接させる
- 状態遷移を線形かつ名前付きにする
- ローカル推論を保全する慣習を選好する

これは言語部族主義よりも強く、かつ実践的な結論である。

10 含意

10.1 言語設計者への含意

これらの結果が引き続き再現される場合、言語設計には新しい問いが必要になる：この言語はローカルビューに何を強制的に表示するか？即座の設計教訓はシンプルである。

1. **契約を第一級にする**。戻り値の慣習、無効入力時の振る舞い、副作用は近接する構文または近接するドキュメントから可視であるべきである。
2. **ドキュメントを開発ループに統合する**。検証済みのサンプルは装飾的なサンプルよりも長持ちする。
3. **隠れた状態よりも可視的なフローを選好する**。余分なミューテーションはそれぞれ、モデルが暗黙的に記憶しなければならない追加事項である。
4. **ドキュメントアーキテクチャを言語アーキテクチャとして扱う**。エコシステムが説明、公開、検索を標準化すれば、モデルはより規則的な世界を見る。

10.2 ソフトウェアチームへの含意

本論文はまた、直接的な運用レシピも提供する。

- Python または TypeScript において、明示的な戻り値・エラー契約を追加する。
- コードベース全体で一貫したドキュメント形式を使用する。
- 可能な限りサンプルを実行可能に保つ。

- レビュアーがシグネチャ、近接するドキュメント、ローカルな本体から関数を理解できるように設計する。

平たく言えば、人間にもモデルにも宝探しをさせないコードを書くということである。

日本のソフトウェア開発チームにとって、これは特に実践的な教訓を含む。日本語の docstring やコメントであっても、構造化されていれば LLM は情報を抽出できる。重要なのは言語ではなく、情報の構造と近接性なのである。

11 ベンチマークを超えて：BEAM プラットフォームとエージェント的含意

本論文における証拠はコード生成に関するものである：プロンプトが与えられたとき、モデルはどれほど信頼性高く正しいコードを生成するか。しかし Elixir のランタイム—BEAM 仮想マシン—は、ベンチマークスコアだけでは語り尽くせないストーリーを持つ。

11.1 エージェントランタイムとしての BEAM

Elixir が動作する Erlang/OTP プラットフォームは、決して停止してはならない電話交換機のために設計された。その中核的抽象—軽量プロセス（各プロセスのヒープ初期値は約 2 KB である）、プリエンプティブスケジューリング、プロセス単位のガベージコレクション、スーパービジョンツリー、透過的分散—は元来、耐障害性を動機としていた [19]。これらの同じ抽象が、現代の AI エージェントフレームワークの要件と少なくとも示唆的に整合するのである：

- **軽量な分離**。BEAM プロセスは OS スレッドよりも軽量であり、大半の言語ランタイムのグリーンスレッドよりも軽量である。実践的には、非常に多数の並行する隔離されたワーカーをサポートできる。
- **障害の分離と回復**。各プロセスは自身のヒープとクラッシュ境界を持つ。失敗したエージェントは隣接するプロセスを汚染せず、スーパーバイザーが宣言されたポリシーに従ってそれを再起動する。これは「let it crash」哲学であり、個々のツール呼び出しや推論ステップが失敗しうるエージェント的システムの信頼性要件に直接対応する。

- **ホットコードリローディング**。BEAM はシステムを停止せずに実行中のコードをアップグレードすることをサポートする。長時間実行されるエージェントデプロイメントにおいて、これはモデル更新、プロンプト変更、ツール登録をサービス中断なしにロールアウトできることを意味する。
- **第一級プリミティブとしてのメッセージパッシング**。BEAM プロセスは非同期メッセージパッシングを通じてのみ通信する。これは、エージェントがミュータブルな状態を共有するのではなく構造化メッセージを交換するマルチエージェントフレームワークにおけるエージェント間通信パターンと密接に類似する。

Guimarães が観察したように、Erlang が 1986 年に導入したアクターモデルは、AI が今日再発見しつつあるエージェントモデルと強い類縁関係を持つ [18]。スーパービジョンツリーはポリシーエンジンとして読めるし、プロセスメールボックスはエージェントインボックスに似ており、GenServer コールバックプロトコルはツール使用インターフェースに似ている。この並行性は示唆的であるが、決定的ではない。

日本では、Erlang/BEAM の技術は通信インフラの文脈で早くから注目されてきた。NTT をはじめとする日本の通信企業が求めた高可用性・高信頼性の要件は、まさに BEAM が設計された文脈そのものである。エージェント的 AI の時代において、この技術的遺産が新たな価値を持つ可能性がある。

11.2 可読性からライブネスへ

この接続は可読性の物語にフィードバックする。モデルにとって生成しやすく、かつ生成されたエージェントをデプロイ、隔離、回復できるランタイムを提供する言語は、複合的優位を享受しうる。Elixir をコード生成モデルにとって読みやすくするドキュメントパイプラインは、生成されたコードがエージェントとして実行される際にも自己文書化をもたらす。`@moduledoc` と `@doc` 属性は本番環境まで生き残り、`Code.fetch_docs/1` によってエージェントが自身または互いのインターフェースを実行時にイントロスペクトすることを可能にする [10]。

11.3 本論文が主張しないこと

本論文は Elixir が唯一の実行可能なエージェントランタイムであるとも、BEAM の特性がここで観察されたベンチマークスコアを引き起こしたとも主張しない。ベンチマーク評価は純粋にプロンプトからのコード生成に関するものであり、デプロイメント、オーケストレーション、障害回復をテストしていない。主張は、同じ設計哲学—意図を可視にし、状態を明示的に保ち、隔離境界を強制する—がコード生成においては可読性として、ランタイム実行においては信頼性として顕現するということである。これらは同一のコインの表裏である。

12 限界と今後の課題

本論文は一つの点において強力であり、他の複数の点において意図的に控えめである。

1. **単一モデルの限界**。完了した全評価は GPT-5.4 の中推論モードを使用している。強い結果はモデルファミリーの変動を生き残るべきである。
2. **ベンチマーク固有性**。AutoCodeBench は大規模で有用であるが、依然として一つのタスク生成パイプラインを持つ一つのベンチマークに過ぎない。
3. **メインリーダーボードにおける共有タスク ID の不在**。完全一致タスクパネルは有用であるが、真に共有されたタスクの多言語ベンチマークを完全には代替しない。
4. **マッチド研究の小規模性**。144 行パネルと 384 行要因計画は有益であるが、3,920 タスクのベンチマークに比べれば依然として小規模である。
5. **プロキシメトリックは近似である**。制御フローと可変性の要約は記述的な道具であり、言語の本質の直接的測定ではない。

したがって、今後の研究は 3 つのことを行うべきである。完全一致タスク多言語パネルのスケールアップ、他のフロンティアモデルファミリーでの能動的アブレーション順序の再現、およびベンチマークスニペットからプロダクションフレームワーク間のリポジトリスケール比較への移行（例えば、Phoenix や Ecto と、Django、Rails、Express との比較）である。

新しい言語設計への含意。 おそらく本研究の最も帰結的な含意は、まだ存在しない言語について示唆することである。LLM 支援プログラミングにおける従来の仮定は、コーパスサイズが生成品質を決定するというものであった。Elixir の結果はこの仮定に直接的に挑戦する。極小の市場シェアを持つ言語が、公開コードコーパスにおいて最も多く表現される言語のスコアをほぼ倍増させている—モデルがより多くの Elixir を暗記したからではなく、Elixir の設計がローカルなコンテキストから意図を回復可能にするからである。

拘束条件がコーパス量ではなく構造的可読性であるなら、新しいプログラミング言語の設計空間は従来考えられていたよりも広い。本論文で特定された特性—明示的契約、パターン駆動制御フロー、第一級アーティファクトとしての構造化ドキュメント、パイプライン指向データ変換、デフォルトイミュータブルセマンティクス—に基づいて一から設計された言語は、原理的には Elixir の LLM 可読性と同等かそれ以上を達成しうる。BEAM プラットフォームの物語（第11節）はさらなる次元を追加する：これらの生成親和的な設計選択をエージェントネイティブなランタイムと対にすることで、優位性が複合される可能性がある。[1]目的設計された LLM 可読言語が進化したエコシステムを最終的に上回るかどうかは、本論文が回答できない実証的問いであるが、ここに提示されたデータは、その問いを真剣に問う価値があることを示唆している。

13 結論

本研究は印象的な結果から出発した。Elixir は AutoCodeBench で 87.4% をスコアし、Python は 43.9% であった。

人工物の統制と因果的追加実験を経て、最も明瞭な解釈が可視となった。この結果は関数型プログラミングについての単純な道徳劇ではない。それは可読性についてのものである。Elixir が異常に高い性能を示すのは、関数の目的、契約、データフロー、失敗モードが近接するコンテキストから読み取り可能であることが多いからである。その構造が除去されると性能は崩壊する。明示的契約がより弱いエコシステムに追加されると性能は上昇する。

本論文の持続的な主張は、したがって Elixir よりも大きく、誇大宣伝よりもシャープである。*LLM* 指向のソフトウェアエンジニアリングは明示性を報酬として与える。予測負荷を軽減する言

[1]著者はこれらの設計上の問いの一部を DreamLang（https://dreamlang.dev）において別途探求している。この注記は関連する進行中の作業の開示として含まれており、本研究における評価対象のアーティファクトとしてではない。

語とエコシステムは、モデルにとって生成、修復、レビュー、拡張がより容易になる。

さらに、BEAM 仮想マシンの軽量プロセス、障害隔離スーパービジョンツリー、ホットコードリローディングは、ここでは直接テストされていないものの、生成だけでなくデプロイメントについてもより広い仮説を動機づける。モデルにとって生成しやすく、かつ生成されたエージェントをデプロイ、隔離、回復できるプラットフォームを提供する言語は、エージェント的システムにおいて複合的優位を享受しうる。

その意味で、将来の言語戦争は速度、安全性、人気だけでは決まらないかもしれない。意図の回復を最も容易にする者が—そしてそれに基づいて行動するエージェントに最適な運用上の住処を提供するランタイムが—勝利するかもしれないのである。

* * *

データとコードの利用可能性

本研究で参照されるすべてのタスク生成スクリプト、プロンプトテンプレート、実行ハーネス、生の評価ログ、および手動監査パックは以下で利用可能である：

https://github.com/gunta/AutoCodeBenchmark

上記リポジトリには、分割要因計画行列、タスク単位の合否 CSV、および本論文のすべての表とプロットを再現するために必要な図生成コードが含まれている。

謝辞

本研究の実施にあたり支援をいただいた株式会社サイバーエージェントの同僚諸氏に感謝する。また、日本の Elixir コミュニティ、特に tokyo.ex および ElixirConf JP の参加者との議論が本研究の着想に貢献したことを付記する。

A 全言語プロキシデータ

Table 9. 全 20 言語のプロキシメトリック(Pass@1 順にソート)。

言語	Pass@1	CF 値	可変性	ドキュメント	パターン	代入数	難問率
Elixir	87.4	6.517	4.289	17.694	3.545	4.970	70.2
Kotlin	76.5	-1.285	11.780	26.144	0.000	10.665	43.5
C#	72.4	-1.196	19.964	30.210	0.000	20.693	46.2
Ruby	63.0	-0.678	11.297	19.840	0.000	11.265	54.0
Julia	57.0	-2.208	9.060	18.166	0.000	9.540	62.5
Dart	56.5	-0.698	14.050	28.481	0.000	13.635	68.0
R	54.5	-1.588	5.844	18.255	0.000	5.389	64.1
Java	51.1	-1.322	14.851	35.082	0.000	12.410	56.4
Racket	51.0	0.103	1.897	15.944	0.000	1.051	56.6
Scala	50.8	1.553	17.202	29.427	0.000	17.653	68.8
Shell	50.5	-0.555	7.320	25.757	0.000	7.580	61.2
C++	50.0	-2.959	29.284	35.156	0.000	23.704	63.4
TypeScript	49.2	-0.676	12.308	11.356	0.000	10.271	75.4
Perl	44.5	-1.326	7.281	15.299	0.000	7.155	61.5
Python	43.9	-2.014	10.671	21.264	0.000	10.020	68.4
Swift	43.5	-1.945	9.434	16.783	0.000	10.220	60.5
Go	42.9	-0.194	9.821	16.338	0.000	9.885	61.3
JavaScript	42.9	-0.574	13.891	15.844	0.000	11.190	64.1
Rust	40.2	-2.718	22.566	26.343	0.000	19.070	76.9
PHP	35.7	-1.217	17.849	11.497	0.000	18.698	55.8

B 能動的アブレーション要約

Table 10. スイート A, D, E, F にわたる能動的アブレーション条件。

スイート	条件	合格	合計	Pass@1	ベースラインとの差
A	full_docs	167	198	84.3%	—
A	reference_no_examples	167	198	84.3%	0.0
A	minimal_docs	91	198	46.0%	-38.4
A	signature_only	85	198	42.9%	-41.4
D	baseline	167	198	84.3%	—
D	case_with	161	198	81.3%	-3.0
D	cond_if	172	198	86.9%	+2.5
D	function_heads	164	198	82.8%	-1.5
E	baseline	166	198	83.8%	—
E	tagged_tuple_helpers	172	198	86.9%	+3.0
E	sentinel_helpers	166	198	83.8%	0.0
F	baseline	162	198	81.8%	—
F	immutable_pipeline	162	198	81.8%	0.0
F	explicit_state_threading	172	198	86.9%	+5.1
F	rebinding_stepwise	173	198	87.4%	+5.6

C マッチド研究のロバストネスチェック

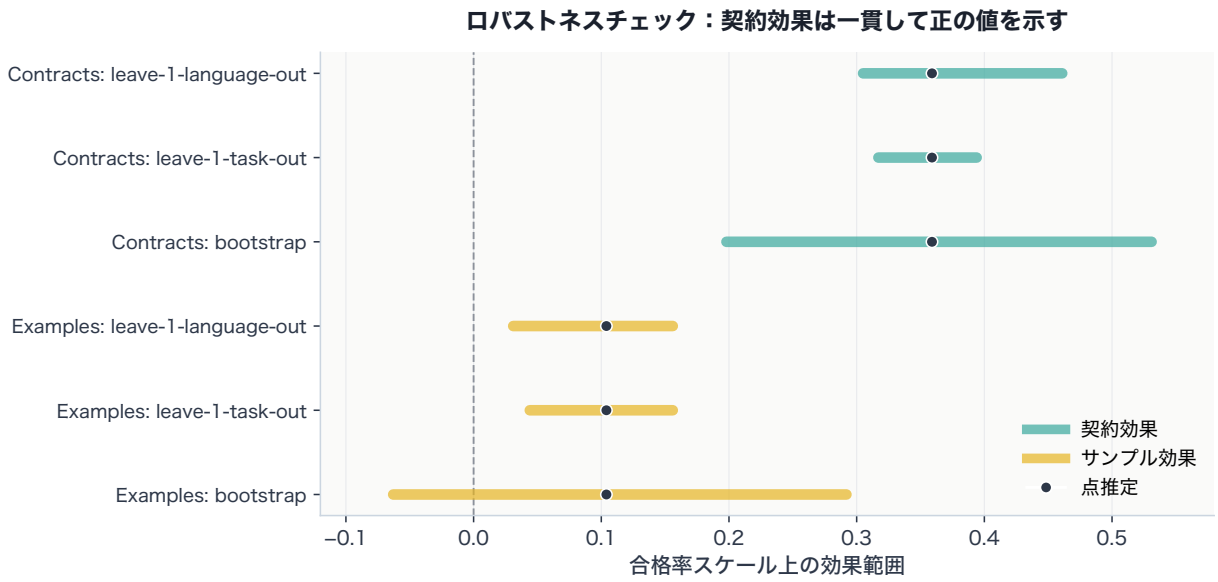

Figure 7. マッチド研究のロバストネス要約。サンプル効果は控えめで低検出力のままである。契約効果はブートストラップおよび一要素除外摂動の両方で正を維持する。

16 タスク明示パネルについて、`rich_contract_examples` から `baseline_compact` を引いたペアードデルタは +0.104 であり、タスククラスターブートストラップ 95% 区間は [−0.063, +0.292] である。一タスク除外デルタは +0.044 から +0.156 の範囲であり、一言語除外デルタは +0.031 から +0.156 の範囲である。

384 行要因計画について、主効果のロバストネス範囲は以下の通りである：

- `docs_rich`：ブートストラップ [−0.021, +0.125]；一タスク除外 [+0.028, +0.061]
- `examples`：ブートストラップ [+0.089, +0.323]；一タスク除外 [+0.172, +0.228]
- `contracts_explicit`：ブートストラップ [+0.198, +0.531]；一タスク除外 [+0.317, +0.394]；一言語除外 [+0.305, +0.461]
- `state_guidance`：ブートストラップ [−0.099, +0.031]；一タスク除外 [−0.033, −0.006]

D 16 タスクパネルのタスクレベル要約

Table 11. 16 タスク明示パネルにおける結果パターン。

タスク	結果パターン
Account Balance Rollup	Elixir は 3 条件すべてで失敗；Python と TypeScript は 3 条件すべてで合格
Bracket Balance Report	全 3 条件で不変
CSV Split With Quotes	全 3 条件で不変
Dense Rankings	Elixir についてサンプルにより救済
Dependency Batches	全 3 条件で不変
Inventory Reconcile	全 3 条件で不変
Merge Touching Intervals	全 3 条件で不変
Normalize Phone Book	Elixir は 3 条件すべてで失敗；Python と TypeScript はサンプル追加時に退行
Queue Wait Times	全 3 条件で不変
Rule Based Discount	Elixir と TypeScript についてサンプルにより救済
Session Durations	全 3 言語についてサンプルにより救済
Stable Group Runs	Elixir についてサンプルにより救済
Threshold Bursts	全 3 条件で不変
Token Bucket Decisions	全 3 言語が全 3 条件で失敗
Top K Frequencies	全 3 条件で不変
Sliding Window Majority	全 3 条件で不変

E 査読者チェックリスト

Table 12. 査読者の懸念事項マップ。

査読者の懸念	本バージョンでの対応	残留状態
メインリーダーボードは観察的	難易度統制＋完全一致タスク補助研究	縮小されたが排除されず
マッチドタスクパネルが小規模すぎる可能性	16 タスク、ペアード検定、タスク付録、ロバストネスチェック	依然として小規模
マッチド結論が単一タスクの人工物である可能性	ブートストラップおよび一要素除外スキャン	実質的に改善
要因計画が脆弱である可能性	Holm 補正＋ロバストネス範囲	範囲限定、普遍的ではない
Elixir のタスクが単に容易である可能性	一言語除外期待値と難問タスク統制	実質的に改善
失敗分析がノイジーである可能性	初回失敗分類法＋リポジトリ内の手動監査パック	部分的に改善
論文が二次的メカニズムを過剰主張する可能性	主張階層テーブルと明示的帰無結果	対応済み

References

[1] Jason Chou, Ao Liu, Yuchi Deng, Zhiying Zeng, Tao Zhang, Haotian Zhu, Jianwei Cai, Yue Mao, Chenchen Zhang, Lingyun Tan, Ziyan Xu, Bohui Zhai, Hengyi Liu, Speed Zhu, Wiggin Zhou, and Fengzong Lian. AutoCodeBench: Large Language Models are Automatic Code Benchmark Generators. *arXiv preprint arXiv:2508.09101*, 2025.

[2] Nguyet-Anh H. Lang, Eric Lang, Thanh Le-Cong, Bach Le, and Quyet-Thang Huynh. Perish or Flourish? A Holistic Evaluation of Large Language Models for Code Generation in Functional Programming. *arXiv preprint arXiv:2601.02060*, 2026.

[3] Sathvik Joel, Jie JW Wu, and Fatemeh H. Fard. A Survey on LLM-based Code Generation for Low-Resource and Domain-Specific Programming Languages. *arXiv preprint arXiv:2410.03981*, 2024.

[4] Micheline Bénédicte Moumoula, Serge Lionel Nikiema, Abdoul Kader Kabore, Jacques Klein, and Tegawendé F. Bissyande. Programming Language Confusion: When Code LLMs Can't Keep their Languages Straight. *arXiv preprint arXiv:2503.13620*, 2025.

[5] Lukas Twist, Jie M. Zhang, Mark Harman, Don Syme, Joost Noppen, Helen Yannakoudakis, and Detlef Nauck. A Study of LLMs' Preferences for Libraries and Programming Languages. *arXiv preprint arXiv:2503.17181*, 2025.

[6] Dashbit. Why Elixir is the Best Language for AI. Dashbit blog, 2025.

[7] Armin Ronacher. A Language for Agents. lucumr.pocoo.org, February 2026. https://lucumr.pocoo.org/2026/2/9/a-language-for-agents/.

[8] T. Florian Jaeger. Redundancy and Reduction: Speakers Manage Syntactic Information Density. *Cognitive Psychology*, 61(1):23–62, 2010.

[9] Elixir Documentation Team. Writing Documentation. HexDocs. https://hexdocs.pm/elixir/writing-documentation.html.

[10] Elixir Documentation Team. Code.fetch_docs/1. HexDocs. https://hexdocs.pm/elixir/Code.html#fetch_docs/1.

[11] ExUnit Documentation Team. ExUnit.DocTest. HexDocs. https://hexdocs.pm/ex_unit/ExUnit.DocTest.html.

[12] ExDoc Documentation Team. ExDoc. HexDocs. https://hexdocs.pm/ex_doc/ExDoc.html.

[13] Hex.pm Team. HexDocs. Hex package manager documentation. https://hex.pm/docs.

[14] Python Documentation Team. pydoc — documentation generator and online help system. Python standard library documentation. https://docs.python.org/3/library/pydoc.html.

[15] Python Documentation Team. doctest — test interactive Python examples. Python standard library documentation. https://docs.python.org/3/library/doctest.html.

[16] TypeScript Team. JSDoc Reference. Official TypeScript documentation. https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html.

[17] TypeDoc Team. Overview. TypeDoc documentation. https://typedoc.org/documents/Overview.html.

[18] George Guimarães. The Actor Model is the Agent Model. Dashbit blog, 2026.

[19] Nguyet-Anh H. Lang, Eric Lang, Bach Le, and Quyet-Thang Huynh. NVLang: A Benchmark for Evaluating LLM Code Generation in Niche Languages on the BEAM. *arXiv preprint arXiv:2512.05224*, 2025.

[20] Zach Daniel. Context-Aware Documentation and LLM Output Quality. Blog post, 2025.